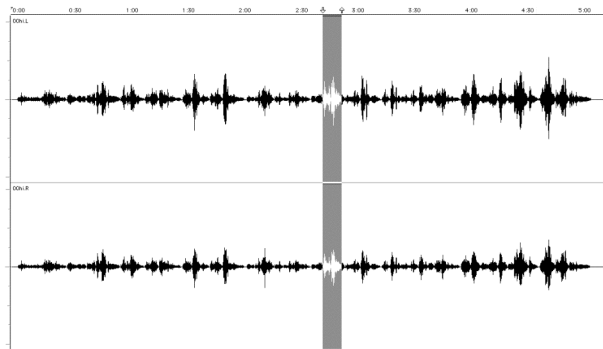
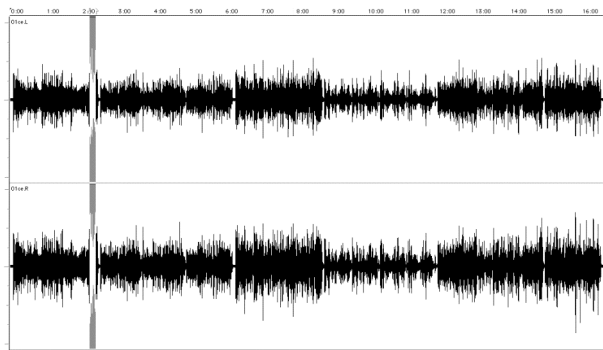
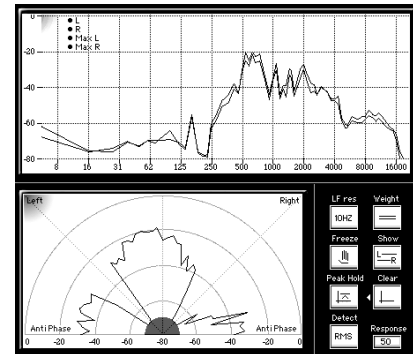


Appendices

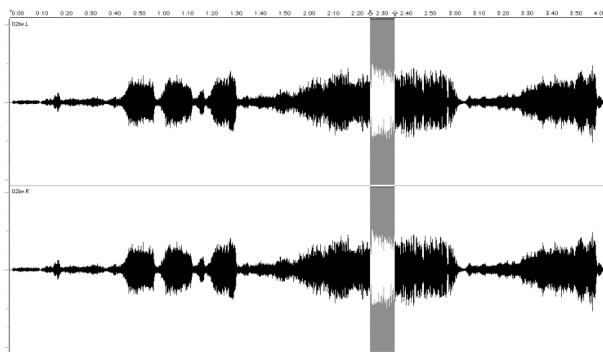
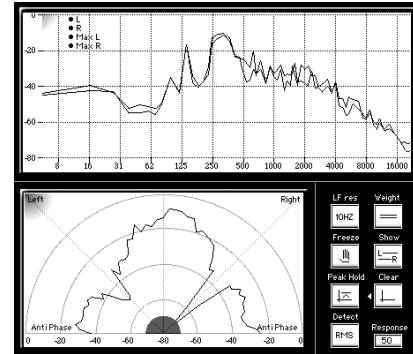
A Screen shots



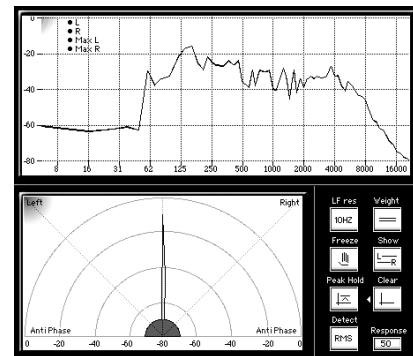
00HI - choral, women's voices

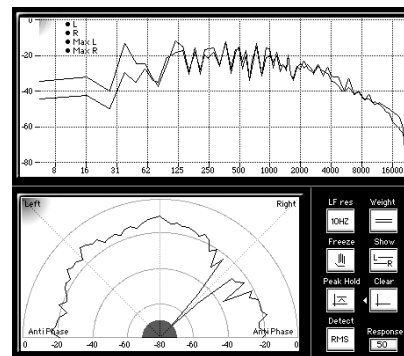
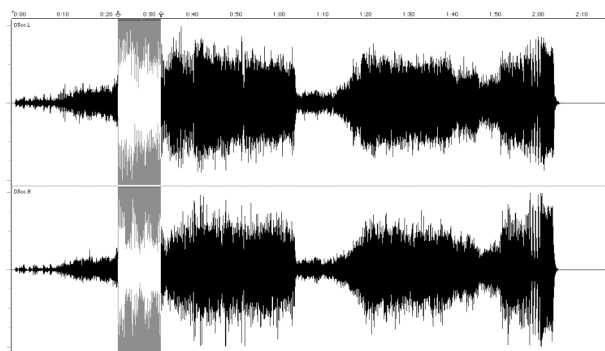


01CE - solo cello, Bach suite

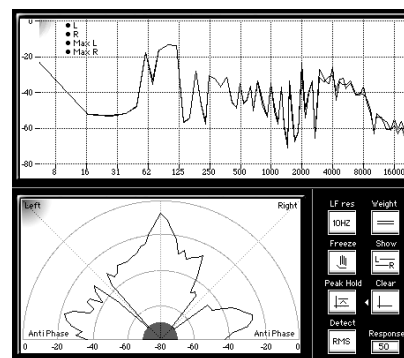
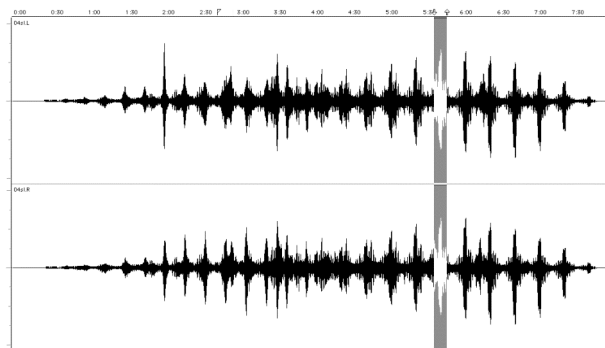


02BE - orchestra symphony, multiple tempos, monophonic

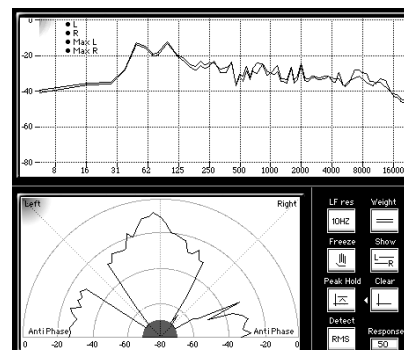
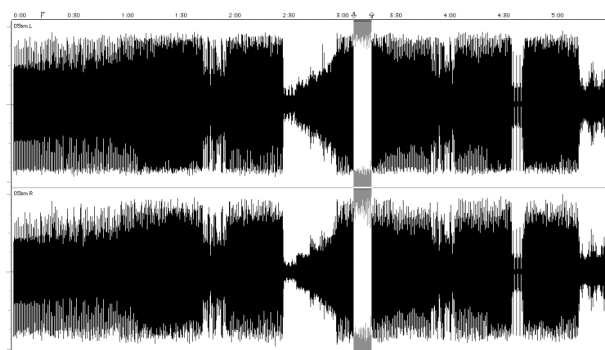




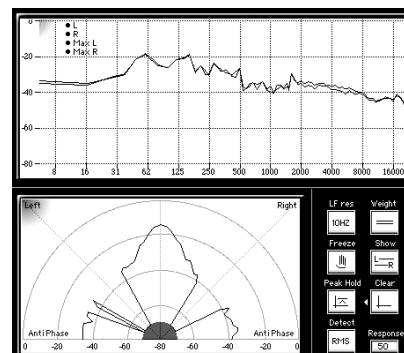
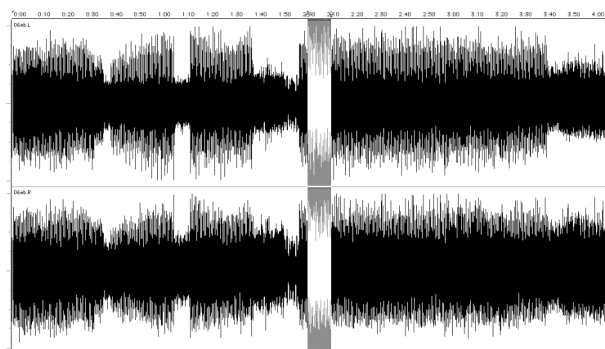
03CC - orchestra ballet, loud and fast



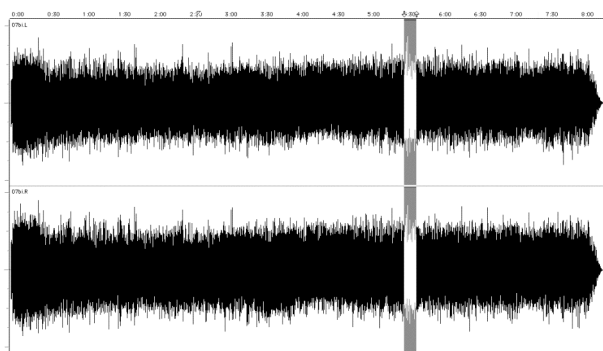
04SL - software synthesis



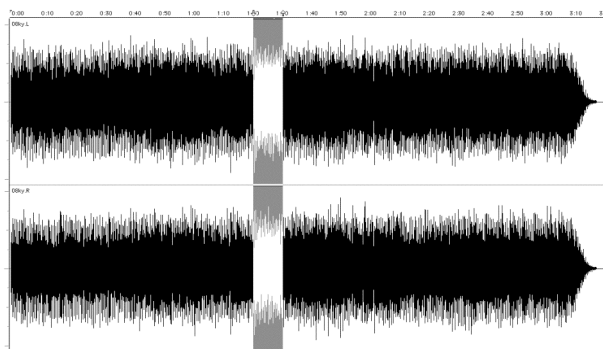
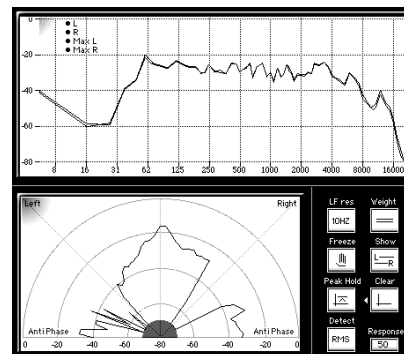
05BM - club techno



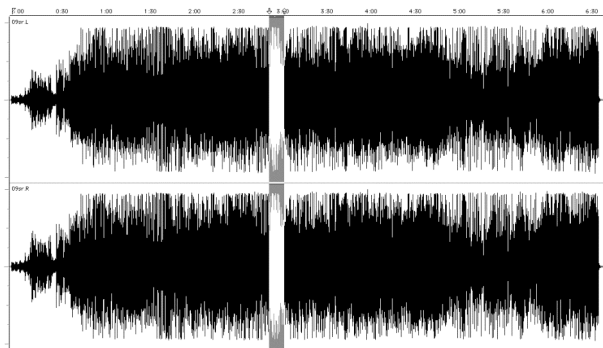
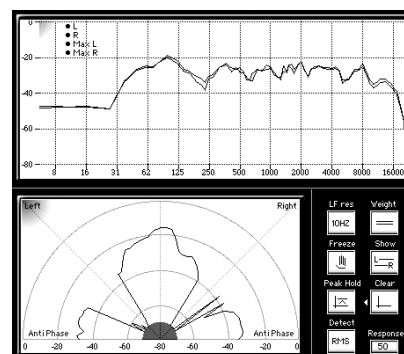
06EB - trance techno



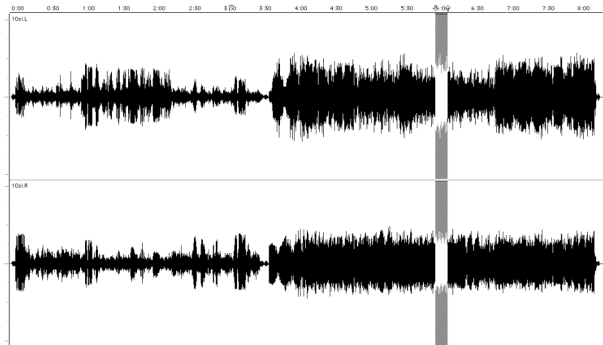
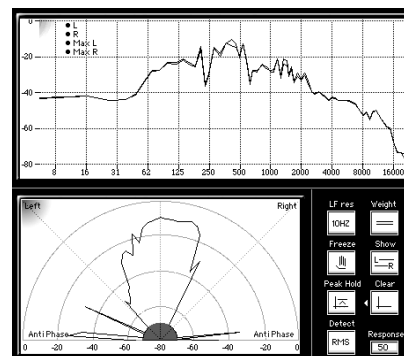
07BI - rock, heavy beat



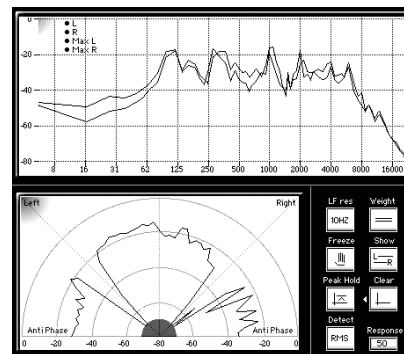
08KY - pop, heavy beat

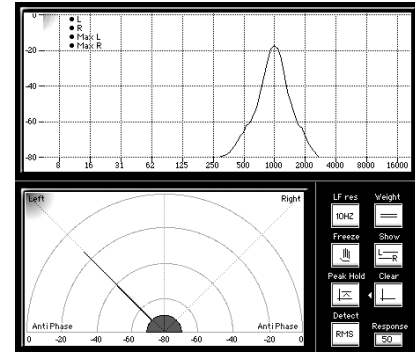
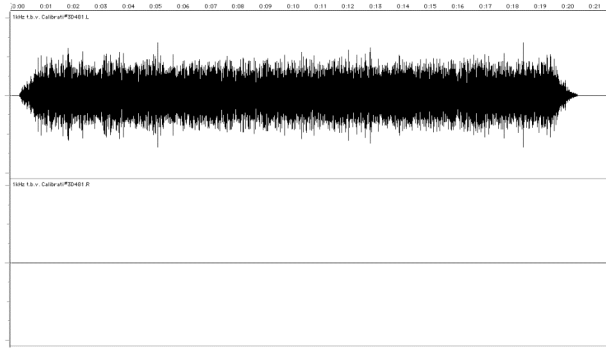


09SR - Indian classical, sitar and tabla

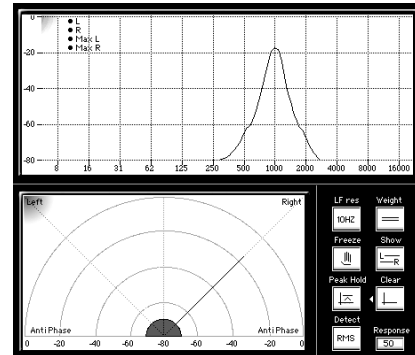
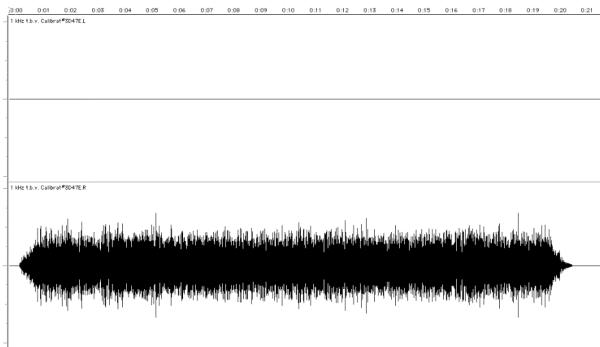


10SI - Indian classical, mandolin and mridangam

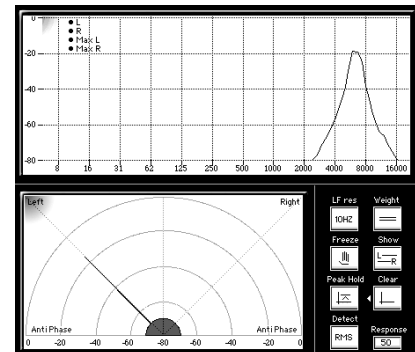
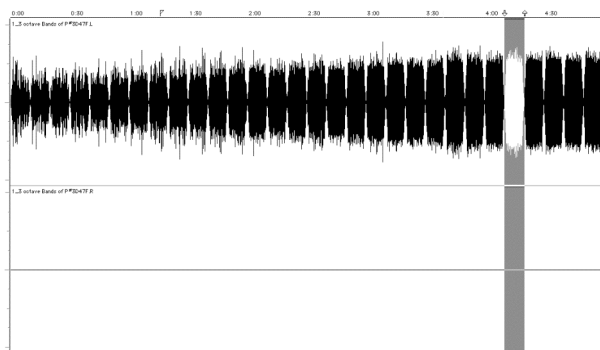




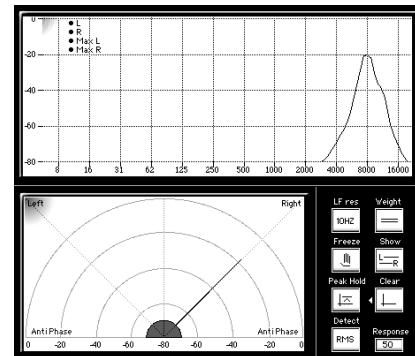
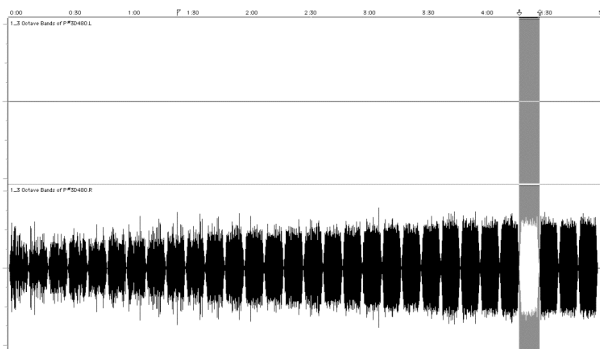
1 kHz t.b.v. calibration - left channel



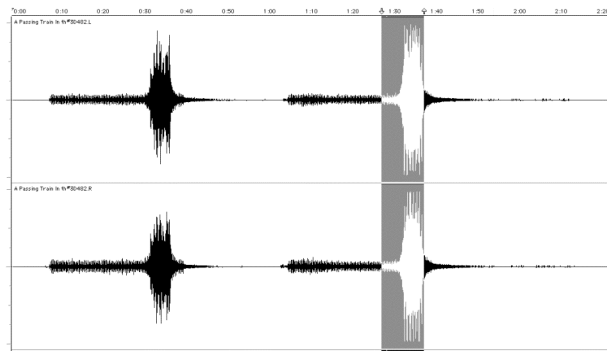
1 kHz t.b.v. calibration - right channel



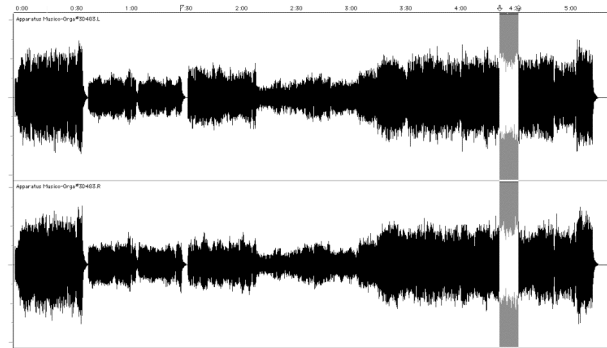
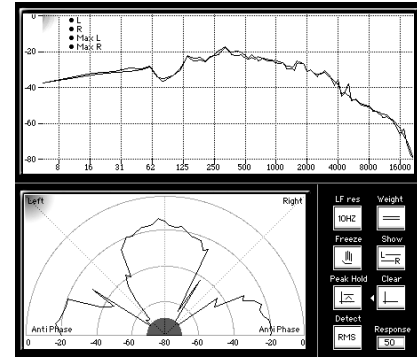
1/3 octave bands of pink noise - left channel



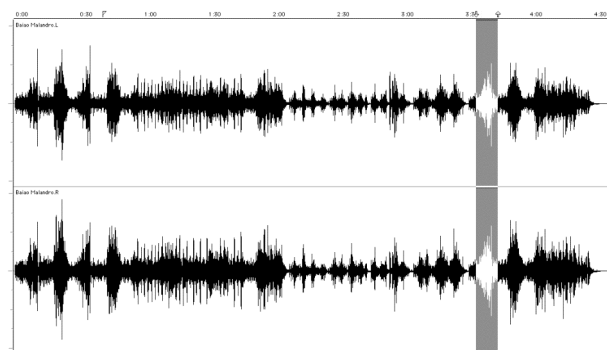
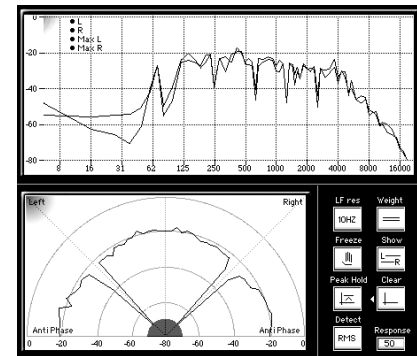
1/3 octave bands of pink noise - right channel



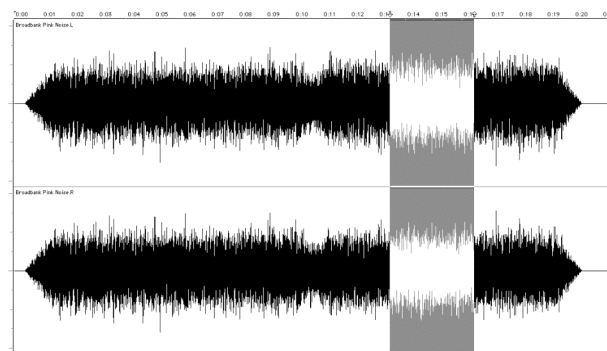
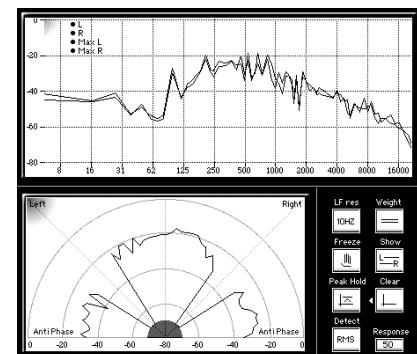
A Passing Train In the Quiet Dutch Farmlands - with crossing bell



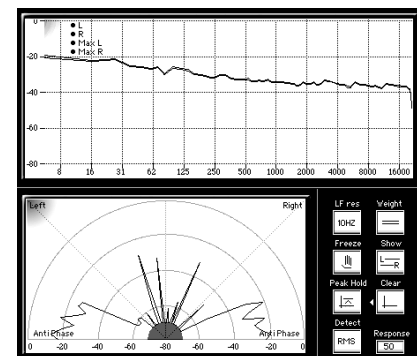
Apparatus Musico-Organisticus, Toccata 1a - pipe organ

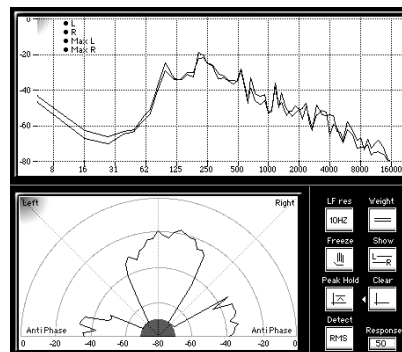
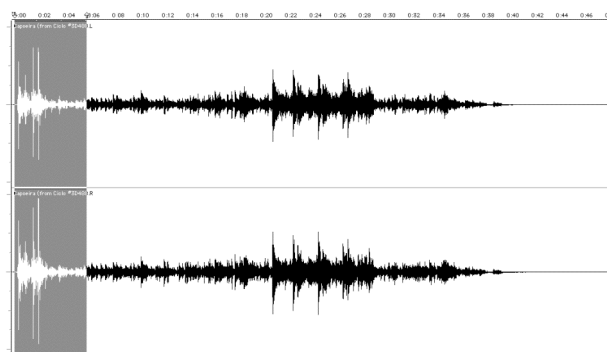


Baiao Malandro - classical guitar duet

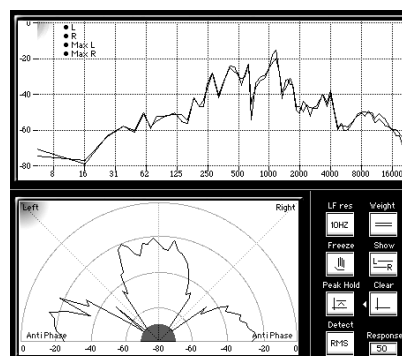
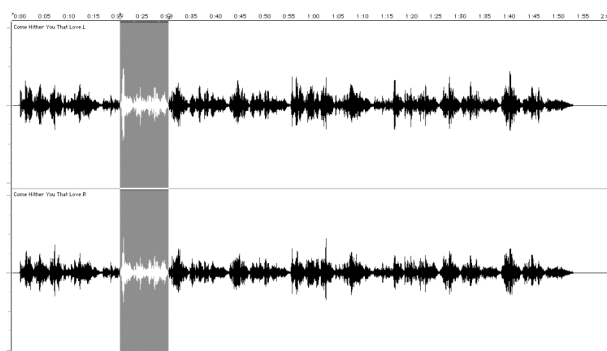


Broadband Pink Noise - second half is out-of-phase

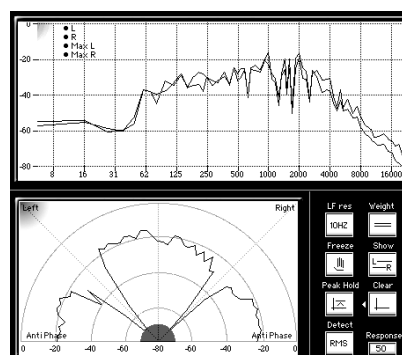
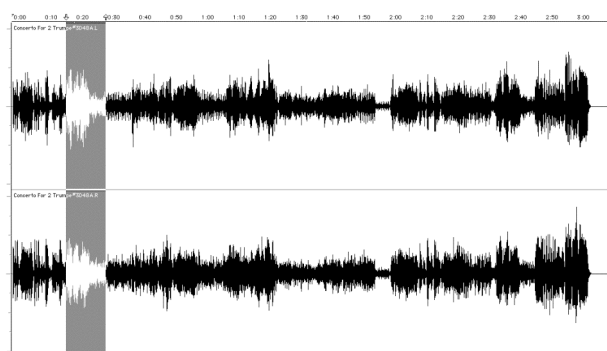




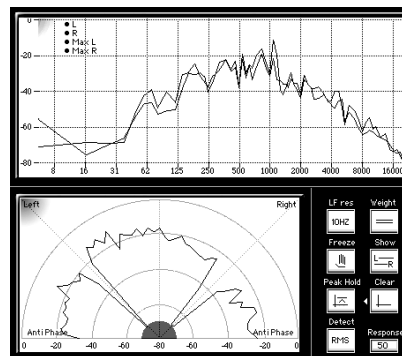
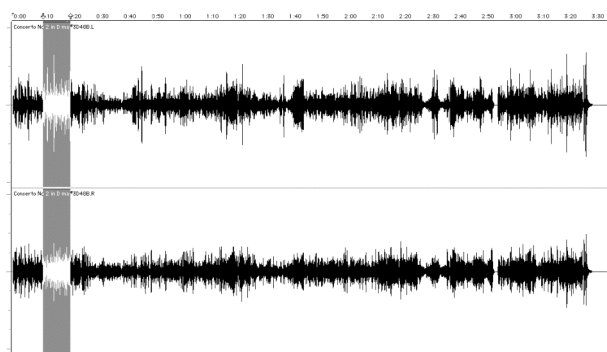
Capoeira (from Ciclo Nordestino No 3) - classical guitar duet, quiet and rhythmic



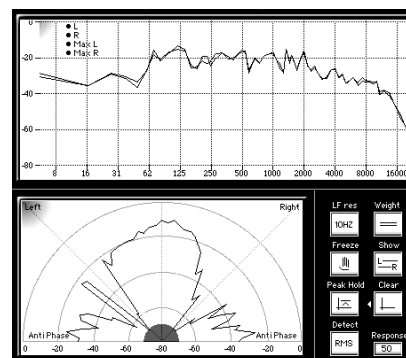
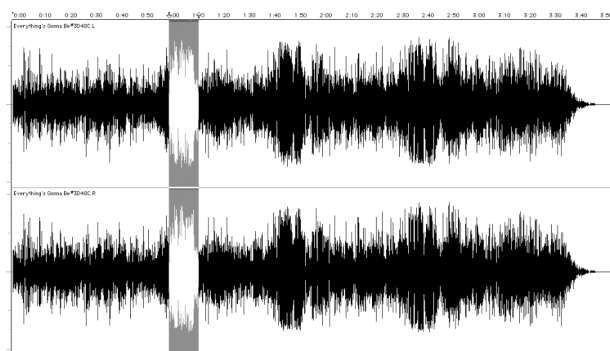
Come Hither You That Love - quiet lute and soprano



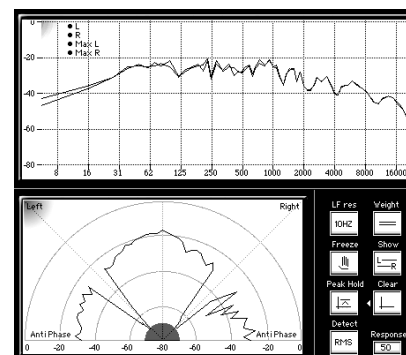
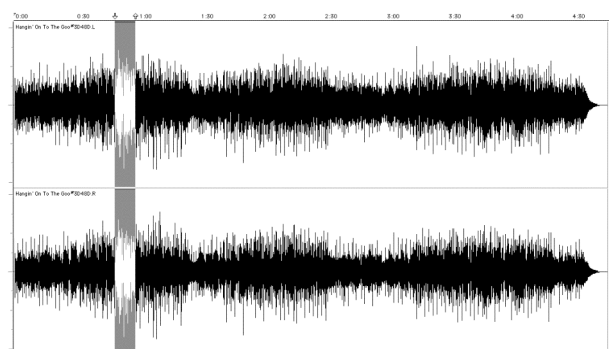
Concerto For 2 Trumpets & Strings, RV 537, C Major, Allegro - baroque



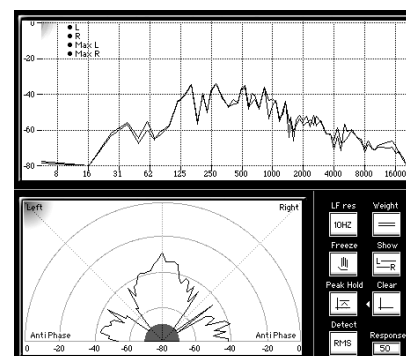
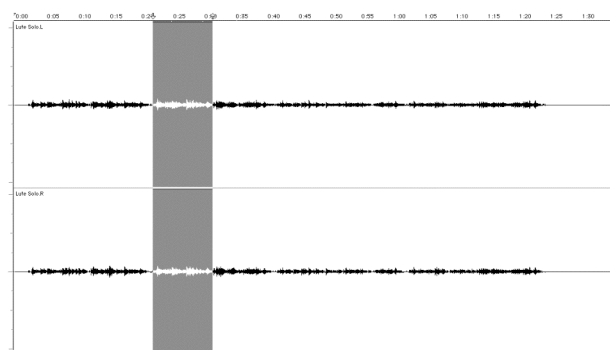
Concerto No 2 in D major, Allegro - baroque flute, harpsichord, and strings



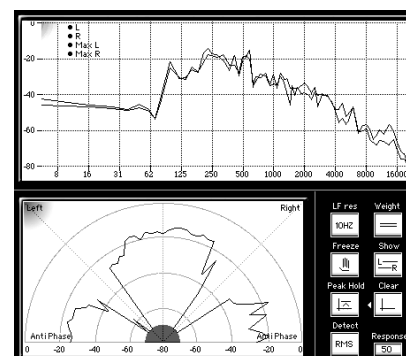
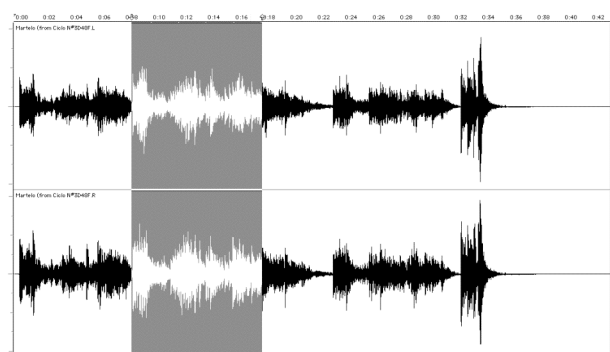
Everything's Gonna Be All Right - electric guitar and woman vocalist



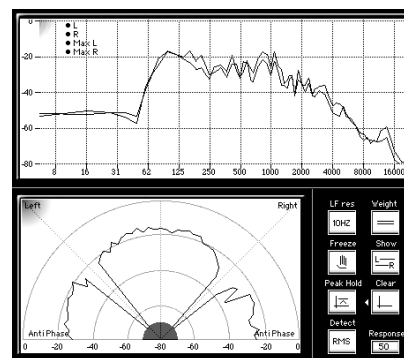
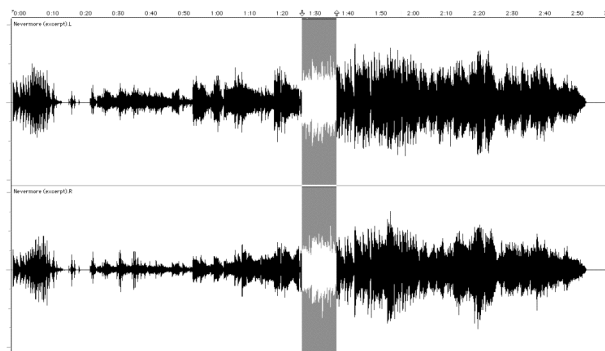
Hangin' On To The Good Times - light pop with male vocalist and heavy bass



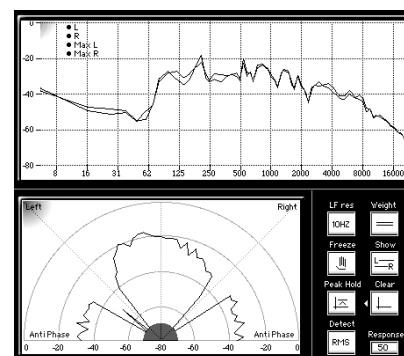
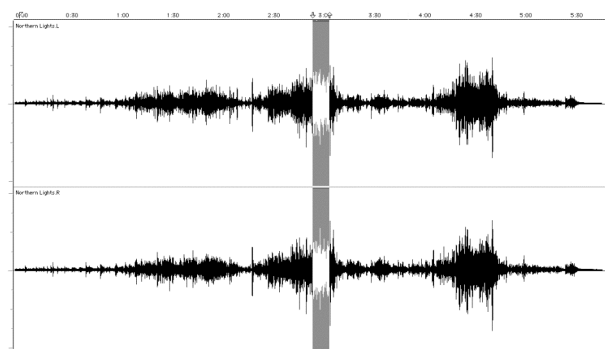
Lute Solo



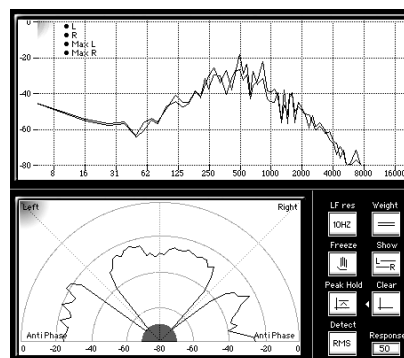
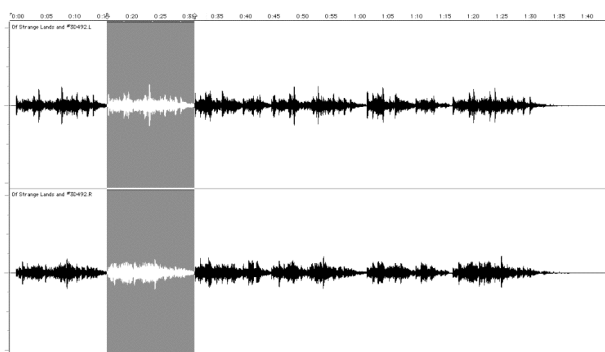
Martelo (from Ciclo Nordestino No. 1) - fast classical guitar duet



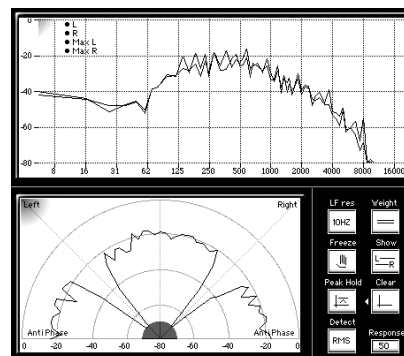
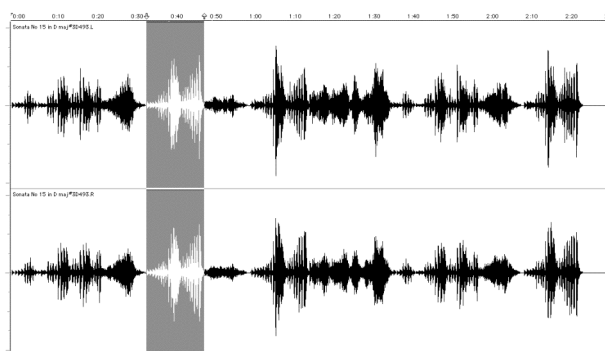
Nevermore (excerpt) - keyboard generated sampled piano, flute, and cello



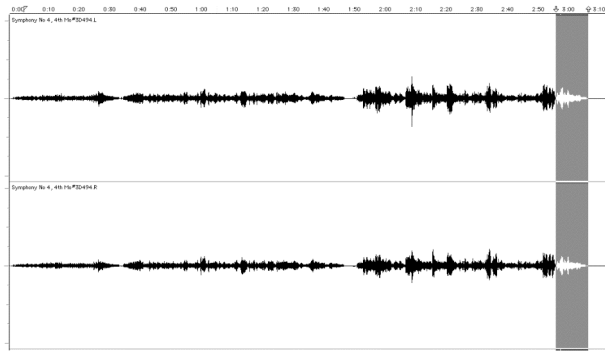
Northern Lights - quiet electric instruments and male vocalist



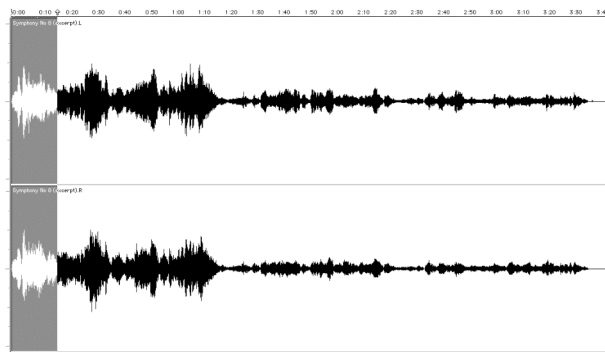
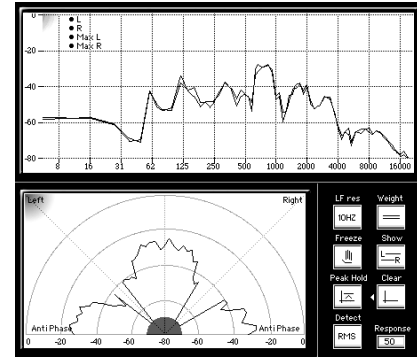
Of Strange Lands and People, Scenes From Childhood, Op. 15 - gentle classical piano solo



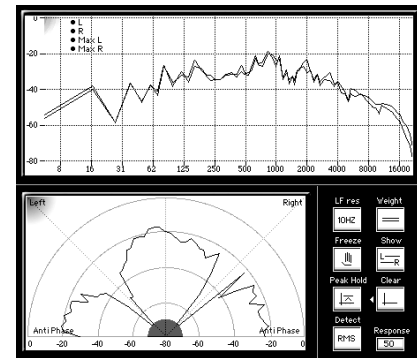
Sonata No 15 in D major, Op. 28 "Pastoral", Scherzo-Allegro Vivace - dynamic classical piano solo



Symphony No 4, 4th Movement (excerpt) - quiet orchestra with female soloist



Symphony No 8 (excerpt) - loud organ, choir, large orchestra; quiets and adds soloists



```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/*
These are the structures used to gather 2 bits at a time.
*/

#ifndef _H_RWlc2bit
#define _H_RWlc2bit
#pragma once

const SInt32 chunkMult    = 8 * 4096; // =32768, should be 4k aligned for best performance on OSX

const SInt32 maxBlock = 32768;          // 2^15; This is the number used to govern the maximum size
                                         // of the "block".

// All values are converted to a long int before examination.
//const long columnMask = 0b00000000000000000000000000000011;
const SInt32 columnMask = 3;
const SInt32 maskSize = 2;

#pragma options align=mac68k

typedef struct zeroRepeat {
    UInt16 usePattern:1; // 0 here means to use this whole structure (don't use pattern)
    UInt16 count:15;     // amount to use '00'; 0 means one, 1 means two, 2 means three, etc., max=32768
} zeroRepeat;           // Counts of <=16 will never be used. Those will fit in bitRepeat.

typedef struct bitRepeat {
    Byte usePattern:1; // 1 here means to use this whole structure (use a pattern)
    Byte useLiteral:1; // 0 here means to use the next two bits as pattern
    Byte pattern:2;    // bit pattern to repeat; 00 01 10 11
    Byte count:4;      // amount to use pattern; 0 means one, 1 means two, 2 means three, etc., max=16
} bitRepeat;

//typedef struct bitLiteralData {
//    Byte p1:2;
//    Byte p2:2;
//    Byte p3:2;
//    Byte p4:2;
//} bitLiteralData;    // CAN'T USE IN STRUCT BELOW. Compiler pads struct boundary to even address!!!!

typedef struct bitLiteral {
    Byte usePattern:1; // 1 here means to use this whole structure (must be a one, use pattern)
    Byte useLiteral:1; // 1 here means to use these next members (must be a one, spell out each literally)
    Byte count:6;      // number of literal bytes (4 pairs of bits); 0 means one, 1 means two, 2 means three, etc.
    Byte bits[64];     // up to 64 of these bytes, corresponding to pairs from 256 samples
} bitLiteral;

#pragma options align=reset

// Flag definitions set true for correlating process applied to block.
const char deltaFlag = 0x01 << 0;
const char sumDiffFlag = 0x01 << 1;
const char wpFlag = 0x01 << 2;
const char anotherFlag = 0x01 << 3;
// certainly more to come...

#endif // _H_RWlc2bit

```

C Encode

```

/////////////////////////////////////////////////////////////////
void
RWLcEncode::Process ( FSSpec &inSrcSpec, FSSpec &inDestSpec )
{
    ///////////////////////////////////////////////////////////////////
    // set up source struct and open source file
    SoundFile sourceFile ( inSrcSpec );
    sourceFile.Open(fsRdPerm);

    // set up destination structure
    SoundFile destFile(inDestSpec);

    // check for enough space on destination drive,
    // return with error if not (set flag)
    FileInfoPB srcInfo ( inSrcSpec );
    if ( !destFile.IsSpaceAvailable(srcInfo.GetSize() + 102400) ) // 100k free space
    {
        mDiskFull = true;
        SysBeep(0);
        return;
    }

    // create destination file, same type as source, with QuickTime Player as the creator
    destFile.CreateAndOpen('TVOD', saveAsType,
        sourceFile.SampleSize(), sourceFile.SampleRate(), sourceFile.Channels(), 'RWLc' );

    long sChans = sourceFile.Channels(); // copied out for readability

#ifdef TEXT_OUTPUT
    unsigned long debugCount = 0;
    unsigned long lastIndex = 0;
    bool blitLastSaved = false;
    unsigned long blitDebugCount = 0;
    char theBitString[64];

    ofstream textFile;
    textFile.open("RWLc.txt", ios::ate);
#endif

    ///////////////////////////////////////////////////////////////////
    // calculate buffer sizes for reading from sources and write to destination
    SInt32 framesRead;
    SInt8 *bufferIn = new SInt8[maxBlock * sourceFile.FrameSize()]; // does this automatically clear the space?
    SInt32 *bufferLR = new SInt32[maxBlock * sChans];
    SInt32 *bufferSD = new SInt32[maxBlock * sChans];

    // Worst case says output buffer needs to be 65/64ths (+1.5%) of the input
    // plus 12.5% (per bit for 8-bit) for the correlating algorithms.
    SInt32 outSize = maxBlock * sourceFile.FrameSize() * 2;
    Byte*bufferOut = new Byte[outSize];

    ///////////////////////////////////////////////////////////////////
    // while we can read one block of source data, run process, and write to file
    while ( (framesRead = sourceFile.ReadFrames((void *)bufferIn, maxBlock)) != 0 )
    {
#ifdef TEXT_OUTPUT
        textFile << "\nNew block started (#" << (SInt32)debugCount++ << ").\n";
#endif

        SInt32 *bufferLong = bufferLR; // This will point to one of the above depending on the outcome of SumDiff.

        ///////////////////////////////////////////////////////////////////
        // convert all to 32 bit ints (for now even 8-bit files which only need to be up converted to 16-bit)
        // This is to make room for any additional bit places generated by our correlation algorithms
        // 20-bit samples have the right most 4 bits set to zero. Shift so the smallest magnitude is 1.
        SInt32 sl;
        SInt32 ib=0;
        switch (sourceFile.SampleSize())
        {
            case 8:
                for(sl=0; sl<(framesRead * sChans); sl++)
                    bufferLong[sl] = bufferIn[sl];
                break;

            case 16:
                SInt16 *shortCast = (SInt16 *) bufferIn;

```

```

        for(sl=0; sl<(framesRead * sChans); sl++)
            bufferLong[sl] = shortCast[sl];
        break;

    case 20:
        for(sl=0; sl<(framesRead * sChans); sl++)
        {
            // take the long pointed to by 'ib', then arithmetic shift to the right (keeps only the left 2.5 bytes)
            bufferLong[sl] = *((SInt32 *) (bufferIn+ib)) >> 12;
            ib += 3;
        }
        break;

    case 24:
        for(sl=0; sl<(framesRead * sChans); sl++)
        {
            // take the long pointed to by 'ib', then arithmetic shift to the right (keeps only the left 3 bytes)
            bufferLong[sl] = *((SInt32 *) (bufferIn+ib)) >> 8;
            ib += 3;
        }
        break;

    default:
        // some error report
        break;
}

// This sets the number of bits to examine minus the size of the mask.
// Each massaging of the stream adds a bit significance to each sample.
// We don't need to examine all 32 bits of the temp buffer. The extra bits are just there to catch the overflow.
long maskLimit = sourceFile.SampleSize();

// clear output buffer so bit-wise OR's work
for (SInt32 b=0; b<outSize; b+=8)
    *(UInt64*)&bufferOut[b] = 0LL; // This could leave up to the last 3 bytes uncleared.

// put a 3 zero-byte marker for the start of the block, this pattern never occurs as part of the compression
//*(UInt32*)&bufferOut[0] = 0L; // Actually write 4 zeros. This insures the forth byte is cleared for the flags.
// this was cleared above.

////////////////////////////////////
// Correlating algorithms!

// take running difference on each channel
for ( SInt32 d = (framesRead*sChans)-1; d >= sChans; d-- )
    bufferLong[d] -= bufferLong[d-sChans];
// bufferOut[3] != deltaFlag;
// maskLimit++;

// difference from the last difference plus the last value (like wavpack 'fast')
for ( SInt32 d = (framesRead*sChans)-1; d >= sChans*2; d-- )
    bufferLong[d] -= (2*bufferLong[d-sChans]) - bufferLong[d-(sChans*2)];
for ( SInt32 d = (sChans*2)-1; d >= sChans; d-- )
    bufferLong[d] -= bufferLong[d-sChans];
bufferOut[3] != wpFlag;
maskLimit++;

// Test the sum and difference of stereo files to see if taking this computation will shrink the files.
if ( sChans == 2 )
{
    //UInt64 magLR = 0, magSD = 0;

    for ( UInt32 s = 0; s < framesRead * sChans; s += 2 )
    {
        bufferSD[s] = bufferLR[s] + bufferLR[s+1];
        bufferSD[s+1] = bufferLR[s] - bufferLR[s+1];
        magLR += bufferLR[s] < 0 ? -bufferLR[s] : bufferLR[s];
        magLR += bufferLR[s+1] < 0 ? -bufferLR[s+1] : bufferLR[s+1];
        magSD += bufferSD[s] < 0 ? -bufferLR[s] : bufferLR[s];
        magSD += bufferSD[s+1] < 0 ? -bufferLR[s+1] : bufferLR[s+1];
    }

    if (magSD < magLR)
    {
        bufferLong = bufferSD; // change to point to data in bufferSD
        bufferOut[3] != sumDiffFlag;
        maskLimit++;
    }
}

```



```

        {
            *(SInt16*)&bufferOut[bOutIndex] = (SInt16) pCount-1;    // same as zeroRepeat data type

            textFile << BitString(&bufferOut[bOutIndex], 2, theBitString) << "\t" << pCount << "\n";

            bOutIndex += 2;
            frmIndex += pCount;
            pCount = 0;
        }
        else if ( pCount > 4 || (pCount==4 && !prevLiteralAvailable) )
        {
            bRep.pattern = firstSample >> maskPosition;
            pCount      = pCount>16 ? 16 : pCount;    // limit run of pattern other than '00' to max 16
            bRep.count   = pCount-1;

            bufferOut[bOutIndex] = *((Byte*)&bRep);

            textFile << BitString(&bufferOut[bOutIndex], 1, theBitString) << "\t\t" << pCount << "\n";

            bOutIndex++;
            frmIndex += pCount;
            pCount = 0;
        }
        else// else just save the next 4 literal pairs
        {
            if (prevLiteralAvailable)
                bLit.count++;
            else
            {
                bLit.count      = 0;
                prevLiteralAvailable = true;
            }

            bLit.bits[bLit.count] = (Byte)((bufferLong[(frmIndex*sChans)+ch] & bitMask) >> maskPosition) << 6;
            frmIndex++;
            bLit.bits[bLit.count] |= (Byte)((bufferLong[(frmIndex*sChans)+ch] & bitMask) >> maskPosition) << 4;
            frmIndex++;
            bLit.bits[bLit.count] |= (Byte)((bufferLong[(frmIndex*sChans)+ch] & bitMask) >> maskPosition) << 2;
            frmIndex++;
            bLit.bits[bLit.count] |= (Byte)((bufferLong[(frmIndex*sChans)+ch] & bitMask) >> maskPosition);
            frmIndex++;

            pCount = 0;
        }
    }

    if (prevLiteralAvailable)// copy remaining literal to output buffer
    {
        ::BlockMove(&bLit, &bufferOut[bOutIndex], bLit.count+2);

        textFile << BitString(&bufferOut[bOutIndex], 1, theBitString) << "\n";
        for (SInt32 tf=1; tf<bLit.count+2; tf++)
            textFile << "\t" << BitString(&bufferOut[bOutIndex+tf], 1, theBitString) << "\n";

        bOutIndex += (bLit.count+2);
    }
}

destFile.PutBytes(bufferOut, bOutIndex);
}
destFile.UpdateHeader(sourceFile.GetFrameMarker());    // make the size the same as what we just read

delete bufferIn;
delete bufferLR;
delete bufferSD;
delete bufferOut;
}

#if TEXT_OUTPUT
char* BitString(void *inData, unsigned long inSize, char *ioStr)    // returns ioStr as a C string
{
    ByteMask= 0b10000000;
    Byte*inCast = (Byte *) inData;

    for (unsigned long b = 0; b<inSize; b++)
    {
        for(unsigned long bt = 0; bt<8; bt++)
        {
            ioStr[(b*8)+bt] = (inCast[b] & (mask >> bt)) ? '1' : '0';
        }
    }
}

```

```
    }  
  }  
  ioStr[inSize*8] = 0;  
  return ioStr;  
}  
#endif
```

D Decode

```

/////////////////////////////////////////////////////////////////
void
RWLcDecode::Process ( FSSpec &inSrcSpec, FSSpec &inDestSpec )
{
///////////////////////////////////////////////////////////////// file management ///////////////////////////////////////////////////////////////////
    // set up source struct and open source file
    SoundFile sourceFile ( inSrcSpec );
    sourceFile.Open(FsRdPerm);

    // must be compressed with my algorithm
    if (sourceFile.Format() != 'RWLc')
    {
        SysBeep(0);
        return;
    }

    // set up destination class
    SoundFile destFile(inDestSpec);

    // check for enough space on destination drive,
    // return with error if not (set flag)
    FileInfoPB srcInfo ( inSrcSpec );
    if ( !destFile.IsSpaceAvailable(srcInfo.GetSize() + 102400) ) // 100k free space
    {
        mDiskFull = true;
        SysBeep(0);
        return;
    }

    // create destination file, same type as source, with QuickTime Player as the creator
    destFile.CreateAndOpen('TVOD', saveAsType, sourceFile.SampleSize(), sourceFile.SampleRate(), sourceFile.Channels() );

///////////////////////////////////////////////////////////////// set up memory ///////////////////////////////////////////////////////////////////
    // declare buffers and counters
    SInt32 bytesRead;
    SInt32 framesInBlock;

    SInt32 sChans = sourceFile.Channels(); // copied out for readability

    UInt8 *bufferIn = new UInt8[chunkMult];
    SInt32 *bufferLong = new SInt32[(chunkMult * sChans)+4]; // plus space for bitliteral to write past end
    SInt8 *bufferOut = new SInt8[chunkMult * sourceFile.FrameSize()];

    SInt32 zeroRepeatRun;
    SInt32 literalCount;

    // initialize values
    SInt32 frmIndex = 0;
    states state = firstZero;
    SInt32 ch = 0;
    UInt8 correlation = 0;

    for (SInt32 cl=0; cl<chunkMult * sChans; cl+=2)
        *(SInt64*)&bufferLong[cl] = 0LL;

    // the mask starts with the 2 bits on the right and moves left
    UInt32 maskPosition;
    UInt32 maskLimit;

///////////////////////////////////////////////////////////////// start conversion ///////////////////////////////////////////////////////////////////
    // read source data, run process, and write to file
    while ( (bytesRead = sourceFile.ReadBytes((void *)bufferIn, chunkMult)) != 0 ) // THIS COULD READ PAST THE AUDIO DATA
    {
        // for each byte in bufferIn
        for ( SInt32 b=0; b<bytesRead; b++ )
        {
            switch ( state )
            {
                case firstZero:
                    if ( bufferIn[b] == 0 )
                        state = secondZero;
                    // else still looking for firstZero, something was wrong
                    // (ie. we're not at the start of a block for streaming)
                    break;
            }
        }
    }
}

```

```

case secondZero:
if ( bufferIn[b] == 0 )
    state = thirdZero;
else
    state = firstZero; // if not, start over looking for first zero
break;

case thirdZero:
if ( bufferIn[b] == 0 )
    state = corrFlags; // we've found the three zero bytes, this is the start of a block
else
    state = firstZero; // bad data, start looking for start of block again
break;

case corrFlags:
// copy correlation flags
correlation = bufferIn[b];

// the mask starts with the 2 bits on the right and moves left
maskPosition = 0L;
maskLimit = destFile.SampleSize();

// each correlation algorithm requires one more bit
for ( Byte m=0x01; m>0x00; m<=<1 )
    if (correlation & m)
        maskLimit++;
state = blockLen1;
break;

case blockLen1:
framesInBlock = ((SInt32)bufferIn[b]) << 8; // copy high-byte of short int (clears lower byte)
state = blockLen2;
break;

case blockLen2:
framesInBlock |= ((SInt32)bufferIn[b]); // copy low-byte of short
framesInBlock++; // plus one to make up for offset
state = nextChunk;
break;

case nextChunk:
switch ( bufferIn[b] & 0b11000000 ) // view only top two bits
{
    case 0b00000000: // 00
    case 0b01000000: // 01
        zeroRepeatRun = ((SInt32)bufferIn[b]) << 8; // copy high-byte of short
        state = zeroRep;
        break;

    case 0b10000000: // 10, do run of pattern now
        //for ( long p=0; p<=((bitRepeat*)&bufferIn[b])->count; p++ ) // p<=count is same as p<count+1
        SInt32 theBits = ((SInt32)(bufferIn[b] & 0b00110000) >> 4) << maskPosition;
        for ( SInt32 p=0; p<=((bufferIn[b] & 0b00001111); p++ ) // p<=count is same as p<count+1
        {
            bufferLong[(frmIndex*sChans)+ch] |= theBits;
            frmIndex++;
        }

        state = nextChunk;
        break;

    case 0b11000000:
        literalCount = bufferIn[b] & 0b00111111; // same as ((bitLiteral*)&bufferIn[b])->count
        state = literalRun;
        break;
}
break;

case zeroRep:
zeroRepeatRun |= ((SInt32)bufferIn[b]); // copy low-byte of short
zeroRepeatRun++;
state = nextChunk;

```

```

// we don't need to write the zeros, they're already there
// just move up the frame count
frmIndex += zeroRepeatRun;
break;

case literalRun:
// copy out literal data
bufferLong[(frmIndex*sChans)+ch] |= ((SInt32)( bufferIn[b]          >> 6)) << maskPosition;
frmIndex++;
bufferLong[(frmIndex*sChans)+ch] |= ((SInt32)((bufferIn[b] & 0b00110000) >> 4)) << maskPosition;
frmIndex++;
bufferLong[(frmIndex*sChans)+ch] |= ((SInt32)((bufferIn[b] & 0b00001100) >> 2)) << maskPosition;
frmIndex++;
bufferLong[(frmIndex*sChans)+ch] |= ((SInt32)(bufferIn[b] & 0b00000011)    ) << maskPosition;
frmIndex++;

if ( literalCount == 0 )
    state = nextChunk;
literalCount--;
break;
}

// test to change to new bit column
if ( frmIndex >= framesInBlock /*&& state != blockLen1*/)
{
    frmIndex = 0;
    maskPosition += maskSize;

    // test for new channel
    if ( maskPosition >= maskLimit )
    {
        maskPosition = 0L;
        ch++;

        // If all channels are done then we're at the end of a block.
        if ( ch >= sChans )
        {
            ch = 0;

            // Rotate and complement.
            // if left most byte is a 1 then this will produce an error (bit?)
            for ( SInt32 cr=0; cr<framesInBlock*sChans; cr++)
                bufferLong[cr] = (bufferLong[cr] & 1L) ? ~(bufferLong[cr] >> 1) : bufferLong[cr] >> 1;

            // Go through de-correlation algorithms in reverse order from the encoding.

            // Sum-Diff (only for 2 channel signals [for now]).
            if (correlation & sumDiffFlag)
            {
                switch (sChans)
                {
                    case 2:
                        // do process in place (same buffer)
                        for ( SInt32 sd=0; sd<framesInBlock*sChans; sd+=2) // (s)um-(d)iff
                        {
                            SInt32 sum  = bufferLong[sd];
                            SInt32 diff = bufferLong[sd+1];
                            bufferLong[sd]  = (sum + diff) >> 1; // divide by 2
                            bufferLong[sd+1] = (sum - diff) >> 1;
                        }
                        break;

                    default: // other combinations of channels might be tried for 3 or more channels
                        break;
                }
            }

            // Delta.
            if (correlation & deltaFlag)
                for ( SInt32 id=sChans; id<framesInBlock*sChans; id++) // (i)ndex (d)elta
                    bufferLong[id] += bufferLong[id-sChans];

            SInt32 sl;
            SInt32 ib=0;
            switch (sourceFile.SampleSize())
            {
                case 8:
                    for(sl=0; sl<(framesInBlock * sChans); sl++)

```

```

        bufferOut[s1] = bufferLong[s1];
        break;

    case 16:
        short *shortCast = (short *) bufferOut;
        for(s1=0; s1<(framesInBlock * sChans); s1++)
            shortCast[s1] = bufferLong[s1];
        break;

    case 20:
        for(s1=0; s1<(framesInBlock * sChans); s1++)
        {
            bufferLong[s1] <= 4;
            Byte*longByte = (Byte *) &bufferLong[s1];
            bufferOut[ib++] = longByte[1];          // zero, the high byte is not used
            bufferOut[ib++] = longByte[2];
            bufferOut[ib++] = longByte[3];
        }
        break;

    case 24:
        for(s1=0; s1<(framesInBlock * sChans); s1++)
        {
            Byte*longByte = (Byte *) &bufferLong[s1];
            bufferOut[ib++] = longByte[1];          // zero, the high byte is not used
            bufferOut[ib++] = longByte[2];
            bufferOut[ib++] = longByte[3];
        }
        break;

    default:
        // some error report
        break;
}

destFile.WriteFrames(bufferOut, framesInBlock);

// clear buffer (write zeros) for next block
for (SInt32 cl=0; cl<chunkMult * sChans; cl+=2)
    *(SInt64*)&bufferLong[cl] = 0LL;

state = firstZero;
    }
}
}
}
}

delete bufferIn;
delete bufferLong;
delete bufferOut;
}

```

E Results

name	size	cp time		gzip only		encode		decode		gzip w/ delta		encode		decode	
		size	t/M	size	%					size	%				
00hi.aif Choral	55918906	1.84	0.35	36888938	66.0	11.94	2.26	6.41	1.21	28594286	51.1	17.87	3.38	5.66	1.07
01ce.aif Solo cello	177312682	7.91	0.47	155205456	87.5	28.9	1.73	22.34	1.33	118959287	67.1	42.38	2.53	20.74	1.24
02be.aif Orchestra	43582666	1.48	0.36	35594753	81.7	8.55	2.08	5.13	1.25	30061992	69.0	9.23	2.24	4.74	1.15
03cc.aif Ballet	24395050	0.59	0.26	20857970	85.5	4.37	1.90	2.87	1.25	17807626	73.0	5.13	2.23	2.93	1.27
04sl.aif Software synthesis	85029610	3.54	0.44	67665617	79.6	16.68	2.08	10.41	1.30	51037353	60.0	21.15	2.63	9.09	1.13
05bm.aif Club techno	59136442	1.89	0.34	55418957	93.7	11.42	2.04	7.32	1.31	50434803	85.3	11.61	2.08	6.3	1.13
06eb.aif Rampant trance techno	43998970	1.21	0.29	40950540	93.1	8.71	2.10	2.35	0.57	35629858	81.0	9.48	2.28	4.92	1.18
07bi.aif Rock	88653962	3.46	0.41	81382151	91.5	17.2	2.05	3.89	0.46	72540016	81.6	19.08	2.27	9.92	1.18
08ky.aif Pop	35896330	1.03	0.30	33169246	92.4	7.25	2.14	1.28	0.38	30907717	86.1	7.07	2.08	4	1.18
09sr.aif Indian classical 1	71693770	3.29	0.49	67314627	93.9	14.09	2.08	3.5	0.52	55139377	76.9	15.82	2.34	7.78	1.15
10si.aif Indian classical 2	89411386	3.61	0.43	78853909	88.2	17.87	2.12	4.81	0.57	71570366	80.0	19.7	2.33	9.87	1.17
1 kHz t.b.v. Calibration- Left Channel.aif	3880854	0.12	0.33	2191314	56.5	1.72	4.69	0.29	0.79	1769781	45.6	1.7	4.64	0.22	0.60
1 kHz t.b.v. Calibration- Right Channel.aif	3883064	0.08	0.22	2233364	57.5	1.65	4.50	0.28	0.76	1822309	46.9	1.68	4.58	0.33	0.90
1_3 Octave Bands of Pink Noise.aif	53451468	1.73	0.34	30567068	57.2	21.82	4.32	5.83	1.15	22127746	41.4	17.28	3.42	4.83	0.96
1_3 octave Bands of Pink Noise- Left Channel.aif	53451454	1.95	0.39	30557093	57.2	21.77	4.31	5.75	1.14	22116928	41.4	17.35	3.44	4.79	0.95
A Passing Train In the Quiet Dutch Farmlands???.aif	25316894	0.30	0.13	16826555	66.5	5.14	2.15	2.84	1.19	12557140	49.6	6.39	2.67	2.1	0.88
Apparatus Musico-Organisticus, Toccata 1a.aif	57685084	2.00	0.37	50367829	87.3	11.17	2.05	7.39	1.36	43914047	76.1	13.21	2.42	6.43	1.18
Baiao Malandro.aif	49217864	2.23	0.48	41742152	84.8	10.12	2.18	2.28	0.49	31319824	63.6	12.45	2.68	5.36	1.15
Broadbank Pink Noise.aif	3765444	0.08	0.22	3319749	88.2	0.77	2.16	0.23	0.65	3217274	85.4	0.81	2.28	0.36	1.01
Capoeira (from Ciclo Nordestino No 3).aif	8563564	0.13	0.16	6223917	72.7	1.69	2.09	0.46	0.57	4496600	52.5	2.3	2.84	0.81	1.00
Come Hither You That Love.aif	21523102	0.27	0.13	17340610	80.6	4.45	2.19	0.77	0.38	14164229	65.8	5.47	2.69	2.22	1.09
Concerto For 2 Trumpets & Strings, RV 537, C Maj	33694726	1.59	0.50	29207135	86.7	6.63	2.08	1.59	0.50	25096246	74.5	7.84	2.46	3.59	1.13
Concerto No 2 in D major, Allegro.aif	38281084	1.13	0.31	32754992	85.6	7.68	2.12	1.3	0.36	26890786	70.2	9.22	2.55	4.25	1.18
Everything's Gonna Be All Right.aif	41545622	1.24	0.32	37282907	89.7	8.32	2.12	1.42	0.36	30921090	74.4	9.47	2.41	4.47	1.14
Hangin' On To The Good Times.aif	50805440	4.95	1.03	45637738	89.8	9.8	2.04	3.01	0.63	37989128	74.8	11.66	2.43	5.61	1.17
Lute Solo.aif	16760254	1.59	1.00	11448268	68.3	3.44	2.17	1.09	0.69	8044680	48.0	4.79	3.02	1.58	1.00
Martelo (from Ciclo Nordestino No. 1).aif	7674506	0.66	0.91	5907548	77.0	1.54	2.12	0.41	0.57	4525428	59.0	1.91	2.63	0.73	1.01
Nevermore (excerpt).aif	32107062	3.45	1.14	26468462	82.4	6.42	2.12	1.08	0.36	22248495	69.3	8.14	2.68	3.36	1.11
Northern Lights.aif	62624250	6.73	1.14	49307363	78.7	12.89	2.18	3.01	0.51	36578243	58.4	16.68	2.82	6.08	1.03
Of Strange Lands and People, Scenes From Childh	18611324	1.70	0.97	14463127	77.7	3.72	2.12	0.81	0.46	10620759	57.1	4.96	2.82	1.78	1.01
Sonata No 15 in D major, Op. 28 _Pastoral_, Schei	26815208	2.65	1.05	21798999	81.3	5.31	2.10	0.95	0.37	16169031	60.3	6.77	2.67	2.55	1.01
Symphony No 4, 4th Movement (excerpt).aif	34482630	3.67	1.13	26018813	75.5	7.31	2.24	1.25	0.38	19958713	57.9	9.29	2.85	3.45	1.06
Symphony No 8 (excerpt).aif	39868698	4.30	1.14	32241100	80.9	8.12	2.16	1.35	0.36	26258917	65.9	10.21	2.71	4.3	1.14
TOTAL	1.459E+09	72.40	0.53	1.207E+09	82.7	308.46	2.24	113.7	0.82	985490084	67.5	358.1	2.60	155.15	1.13

Table 6a. Results for cp, gzip only, and gzip with delta encoding.

file name	size	slac alone size	%	encode	decode	slac w/delta size	%	encode	decode				
00hi.aif Choral	55918906	35703339	63.8	7.13	1.35	5.67	1.07	30255451	54.1	6.29	1.19	5.28	1.00
01ce.aif Solo cello	177312682	130909484	73.8	22.38	1.34	19.03	1.14	99449984	56.1	20.08	1.20	17.89	1.07
02be.aif Orchestra	43582666	27736777	63.6	5.23	1.27	4.48	1.09	23804354	54.6	4.89	1.19	4.12	1.00
03cc.aif Ballet	24395050	19634100	80.5	3.2	1.39	2.61	1.13	16566626	67.9	2.92	1.27	2.55	1.11
04sl.aif Software synthesis	85029610	57779407	68.0	10.51	1.31	8.78	1.09	46481490	54.7	9.87	1.23	8.25	1.03
05bm.aif Club techno	59136442	52251841	88.4	8.48	1.52	6.83	1.22	47630496	80.5	7.9	1.41	7.11	1.27
06eb.aif Rampant trance techno	43998970	37812467	85.9	5.77	1.39	4.93	1.19	33617040	76.4	5.7	1.37	4.91	1.18
07bi.aif Rock	88653962	76253636	85.8	12.25	1.46	10.07	1.20	67812148	76.3	11.88	1.42	10.03	1.19
08ky.aif Pop	35896330	32362702	90.2	5.17	1.52	4.09	1.21	30067607	83.8	5.08	1.50	4.14	1.22
09sr.aif Indian classical 1	71693770	58937748	82.2	9.09	1.34	8.06	1.19	46982316	65.5	8.6	1.27	7.72	1.14
10si.aif Indian classical 2	89411386	74315469	83.1	12.9	1.53	9.81	1.16	65520794	73.3	12.01	1.42	9.76	1.16
1 kHz t.b.v. Calibration- Left Channel.aif	3880854	1564683	40.3	0.49	1.34	0.19	0.52	1238264	31.9	0.47	1.28	0.2	0.55
1 kHz t.b.v. Calibration- Right Channel.aif	3883064	1616759	41.6	0.51	1.39	0.2	0.55	1300958	33.5	0.47	1.28	0.19	0.52
1_3 Octave Bands of Pink Noise.aif	53451468	20759070	38.8	5.34	1.06	4.53	0.90	16328474	30.5	4.84	0.96	4.34	0.86
1_3 octave Bands of Pink Noise- Left Channel.aif	53451454	20750671	38.8	5.05	1.00	4.11	0.81	16316850	30.5	4.59	0.91	4.23	0.84
A Passing Train In the Quiet Dutch Farmlands???.aif	25316894	14235630	56.2	3.13	1.31	2.28	0.95	11057384	43.7	2.57	1.07	2.09	0.87
Apparatus Musico-Organisticus, Toccata 1a.aif	57685084	48827675	84.6	8.26	1.52	6.27	1.15	41440645	71.8	7.5	1.38	6.36	1.17
Baiao Malandro.aif	49217864	34900972	70.9	6.27	1.35	5.07	1.09	26165276	53.2	5.71	1.23	4.87	1.05
Broadbank Pink Noise.aif	3765444	3264771	86.7	0.49	1.38	0.29	0.82	2977349	79.1	0.51	1.43	0.27	0.76
Capoeira (from Ciclo Nordestino No 3).aif	8563564	5158282	60.2	1.26	1.56	0.79	0.98	3770374	44.0	0.95	1.17	0.6	0.74
Come Hither You That Love.aif	21523102	15204020	70.6	2.64	1.30	2.06	1.01	12348747	57.4	2.35	1.16	1.99	0.98
Concerto For 2 Trumpets & Strings, RV 537, C Maj	33694726	26323419	78.1	4.61	1.45	3.68	1.16	22363043	66.4	4.21	1.32	3.54	1.11
Concerto No 2 in D major, Allegro.aif	38281084	28753728	75.1	5.09	1.41	4.15	1.15	23413383	61.2	4.7	1.30	3.87	1.07
Everything's Gonna Be All Right.aif	41545622	32789813	78.9	5.55	1.41	4.35	1.11	27767607	66.8	5.15	1.31	4.38	1.12
Hangin' On To The Good Times.aif	50805440	40470915	79.7	7.19	1.50	5.41	1.13	34082321	67.1	6.38	1.33	4.98	1.04
Lute Solo.aif	16760254	9301722	55.5	1.94	1.23	1.44	0.91	7208986	43.0	1.66	1.05	1.28	0.81
Martelo (from Ciclo Nordestino No. 1).aif	7674506	5073713	66.1	1.1	1.52	0.64	0.88	3822768	49.8	0.87	1.20	0.54	0.74
Nevermore (excerpt).aif	32107062	25075791	78.1	4.24	1.40	3.55	1.17	20455885	63.7	3.93	1.30	3.1	1.02
Northern Lights.aif	62624250	41600324	66.4	8.35	1.41	6.62	1.12	32965446	52.6	6.87	1.16	5.76	0.97
Of Strange Lands and People, Scenes From Childhood	18611324	11967849	64.3	2.16	1.23	1.91	1.09	8518538	45.8	2.05	1.17	1.45	0.82
Sonata No 15 in D major, Op. 28 _Pastoral_, Scherzo	26815208	18100951	67.5	3.35	1.32	2.74	1.08	13228385	49.3	2.99	1.18	2.19	0.86
Symphony No 4, 4th Movement (excerpt).aif	34482630	22141275	64.2	4.28	1.31	3.29	1.01	17762170	51.5	3.81	1.17	3.06	0.94
Symphony No 8 (excerpt).aif	39868698	28433988	71.3	5.05	1.34	3.95	1.05	23016877	57.7	4.6	1.22	3.57	0.95
TOTAL	1.459E+09	1.06E+09	72.6	188.46	1.37	151.88	1.10	875738036	60.0	172.4	1.25	144.62	1.05

Table 6b. Results for SLAC with no decorrelation, and with delta encoding.

file	size	slac w/wpFast	size	%	encode	decode	flac -0	size	%	encode	decode		
00hi.aif Choral	55918906	29051026	52.0	6.34	1.20	5.47	1.04	21047970	37.6	4.05	0.77	5.29	1.00
01ce.aif Solo cello	177312682	89236238	50.3	18.85	1.13	16.87	1.01	75200532	42.4	12.29	0.73	18.05	1.08
02be.aif Orchestra	43582666	23343243	53.6	4.97	1.21	3.98	0.97	24376447	55.9	3.4	0.83	5.07	1.23
03cc.aif Ballet	24395050	15965050	65.4	2.95	1.28	2.45	1.06	14235912	58.4	2.8	1.21	2.58	1.12
04sl.aif Software synthesis	85029610	43664922	51.4	9.26	1.15	7.53	0.94	36325332	42.7	8.5	1.06	8.62	1.07
05bm.aif Club techno	59136442	47726370	80.7	7.72	1.38	6.82	1.22	43949202	74.3	7.18	1.29	7.61	1.36
06eb.aif Rampant trance techno	43998970	33547446	76.2	5.59	1.34	4.74	1.14	30367169	69.0	5.52	1.33	5.35	1.29
07bi.aif Rock	88853962	65331051	73.5	11.38	1.36	9.44	1.12	58032630	65.3	10.59	1.26	10.82	1.29
08ky.aif Pop	35896330	29784524	83.0	4.84	1.43	4.28	1.26	26820677	74.7	4.44	1.31	4.51	1.33
09sr.aif Indian classical 1	71693770	40904081	57.1	7.97	1.18	7.03	1.04	38656259	53.9	7.89	1.16	7.93	1.17
10si.aif Indian classical 2	89411386	60250542	67.4	11.16	1.32	8.77	1.04	51308077	57.4	10.36	1.23	9.9	1.17
1 kHz t.b.v. Calibration- Left Channel.aif	3880854	940130	24.2	0.47	1.28	0.16	0.44	667733	17.2	0.43	1.17	0.36	0.98
1 kHz t.b.v. Calibration- Right Channel.aif	3883064	1009092	26.0	0.42	1.14	0.17	0.46	708914	18.3	0.53	1.44	0.36	0.98
1_3 Octave Bands of Pink Noise.aif	53451468	14045228	26.3	4.77	0.94	3.71	0.73	11658078	21.8	4.36	0.86	4.45	0.88
1_3 octave Bands of Pink Noise- Left Channel.aif	53451454	14032276	26.3	4.6	0.91	3.79	0.75	11651042	21.8	4.23	0.84	4.01	0.79
A Passing Train In the Quiet Dutch Farmlands???.aif	25316894	10540021	41.6	2.54	1.06	1.91	0.80	8877209	35.1	2.66	1.11	2.44	1.02
Apparatus Musico-Organisticus, Toccata 1a.aif	57685084	37664922	65.3	6.91	1.27	5.68	1.04	32508059	56.4	6.69	1.23	6.13	1.12
Baiao Malandro.aif	49217864	21482996	43.6	5.12	1.10	4.06	0.87	18245407	37.1	4.97	1.07	4.79	1.03
Broadbank Pink Noise.aif	3765444	3108558	82.6	0.63	1.77	0.29	0.82	2982954	79.2	0.61	1.71	0.44	1.24
Capoeira (from Ciclo Nordestino No 3).aif	8563564	3120670	36.4	0.83	1.03	0.47	0.58	2682263	31.3	0.92	1.14	0.76	0.94
Come Hither You That Love.aif	21523102	11610634	53.9	2.33	1.15	1.86	0.91	9924867	46.1	2.37	1.17	2.19	1.08
Concerto For 2 Trumpets & Strings, RV 537, C Maj	33694726	19846807	58.9	3.82	1.20	3.1	0.97	16399931	48.7	3.59	1.13	3.56	1.12
Concerto No 2 in D major, Allegro.aif	38281084	20794102	54.3	4.15	1.15	3.45	0.95	17275305	45.1	4.17	1.15	3.8	1.05
Everything's Gonna Be All Right.aif	41545622	26917461	64.8	5.03	1.28	4.19	1.07	24384017	58.7	4.76	1.21	4.6	1.17
Hangin' On To The Good Times.aif	50805440	32863438	64.7	6.46	1.35	4.82	1.00	28817311	56.7	5.55	1.16	5.98	1.25
Lute Solo.aif	16760254	6609844	39.4	1.78	1.12	1.19	0.75	5679886	33.9	1.69	1.07	1.61	1.02
Martelo (from Ciclo Nordestino No. 1).aif	7674506	3223969	42.0	0.71	0.98	0.53	0.73	2805063	36.6	0.87	1.20	0.85	1.17
Nevermore (excerpt).aif	32107062	17654616	55.0	3.85	1.27	2.84	0.94	13801949	43.0	3.35	1.10	3.3	1.09
Northern Lights.aif	62624250	31366390	50.1	6.82	1.15	5.46	0.92	26395824	42.1	6.65	1.12	6.32	1.07
Of Strange Lands and People, Scenes From Child	18611324	6566405	35.3	1.79	1.02	1.31	0.74	5395618	29.0	1.89	1.07	1.69	0.96
Sonata No 15 in D major, Op. 28 _Pastoral_, Sche	26815208	10960762	40.9	2.63	1.04	2.03	0.80	8544867	31.9	2.66	1.05	2.22	0.88
Symphony No 4, 4th Movement (excerpt).aif	34482630	16318208	47.3	3.66	1.12	2.86	0.88	13604149	39.5	3.56	1.09	3.23	0.99
Symphony No 8 (excerpt).aif	39868698	20507520	51.4	4.33	1.15	3.46	0.92	17459269	43.8	4.19	1.11	4.23	1.12
TOTAL	1.459E+09	809988542	55.5	164.68	1.19	134.72	0.98	700789922	48.0	147.72	1.07	153.05	1.11

Table 6c. Results for SLAC with WavPack fast encoding and FLAC with option “-0”.

file name	size	flac -8 size	%	encode	decode		WavPack -f size	%	encode	decode			
00hi.aif Choral	55918906	19781507	35.4	39.75	7.52	5.17	0.98	20625476	36.9	6.48	1.23	5.91	1.12
01ce.aif Solo cello	177312682	73732170	41.6	126.59	7.56	17.16	1.02	73095180	41.2	20.46	1.22	19.04	1.14
02be.aif Orchestra	43582666	17835781	40.9	31.05	7.54	4.41	1.07	18061710	41.4	4.99	1.21	5.13	1.25
03cc.aif Ballet	24395050	13538417	55.5	17.63	7.65	2.7	1.17	13729824	56.3	3.09	1.34	3.18	1.38
04sl.aif Software synthesis	85029610	34691660	40.8	60.3	7.51	8.98	1.12	36537236	43.0	9.77	1.22	9.26	1.15
05bm.aif Club techno	59136442	40256556	68.1	43.11	7.72	7.3	1.31	41265946	69.8	8.2	1.47	7.99	1.43
06eb.aif Rampant trance techno	43998970	29419852	66.9	32.24	7.76	5.44	1.31	29431580	66.9	5.65	1.36	5.79	1.39
07bi.aif Rock	88853962	52726291	59.3	64.38	7.67	10.01	1.19	53929020	60.7	11.08	1.32	11.38	1.36
08ky.aif Pop	35896330	25174854	70.1	26.11	7.70	4.71	1.39	25434892	70.9	4.78	1.41	4.88	1.44
09sr.aif Indian classical 1	71693770	33231706	46.4	51.54	7.61	7.22	1.07	32377318	45.2	9.67	1.43	8.3	1.23
10si.aif Indian classical 2	89411386	46166424	51.6	64.16	7.59	9.72	1.15	46324118	51.8	10.8	1.28	10.43	1.23
1 kHz t.b.v. Calibration- Left Channel.aif	3880854	655037	16.9	2.3	6.27	0.37	1.01	1233968	31.8	0.67	1.83	0.45	1.23
1 kHz t.b.v. Calibration- Right Channel.aif	3883064	696794	17.9	2.3	6.27	0.38	1.04	1246190	32.1	0.64	1.74	0.48	1.31
1_3 Octave Bands of Pink Noise.aif	53451468	9810047	18.4	28.26	5.60	4.22	0.84	20387766	38.1	5.96	1.18	5.5	1.09
1_3 octave Bands of Pink Noise- Left Channel.aif	53451454	9795553	18.3	27.92	5.53	3.63	0.72	20395024	38.2	6.19	1.23	5.45	1.08
A Passing Train In the Quiet Dutch Farmlands???.aif	25316894	8345075	33.0	17.11	7.15	2.12	0.89	8543938	33.7	3.63	1.52	2.56	1.07
Apparatus Musico-Organisticus, Toccata 1a.aif	57685084	30898128	53.6	39.99	7.34	6.25	1.15	30348078	52.6	7.14	1.31	7.11	1.30
Baiao Malandro.aif	49217864	17984231	36.5	33.63	7.23	4.5	0.97	17781630	36.1	5.53	1.19	4.92	1.06
Broadbank Pink Noise.aif	3765444	2552663	67.8	2.83	7.95	0.52	1.46	2617764	69.5	0.61	1.71	0.51	1.43
Capoeira (from Ciclo Nordestino No 3).aif	8563564	2648615	30.9	5.7	7.04	0.8	0.99	2633968	30.8	1.02	1.26	0.97	1.20
Come Hither You That Love.aif	21523102	9568497	44.5	14.65	7.20	2.26	1.11	9691908	45.0	2.66	1.31	2.48	1.22
Concerto For 2 Trumpets & Strings, RV 537, C Maj	33694726	15913373	47.2	23.39	7.35	3.79	1.19	15734670	46.7	4.17	1.31	4.23	1.33
Concerto No 2 in D major, Allegro.aif	38281084	16853072	44.0	26.79	7.41	3.9	1.08	16587168	43.3	4.57	1.26	4.33	1.20
Everything's Gonna Be All Right.aif	41545622	22468194	54.1	28.89	7.36	4.66	1.19	23025398	55.4	5.25	1.34	5.12	1.30
Hangin' On To The Good Times.aif	50805440	27792107	54.7	35.78	7.45	5.5	1.15	28035000	55.2	6.28	1.31	6.55	1.36
Lute Solo.aif	16760254	5489756	32.8	11.25	7.10	1.47	0.93	5602944	33.4	1.98	1.25	1.71	1.08
Martelo (from Ciclo Nordestino No. 1).aif	7674506	2776668	36.2	5	6.90	0.74	1.02	2763548	36.0	0.88	1.21	0.77	1.06
Nevermore (excerpt).aif	32107062	13329995	41.5	21.88	7.21	2.89	0.95	13022718	40.6	3.89	1.28	3.39	1.12
Northern Lights.aif	62624250	24987009	39.9	43.23	7.31	6.63	1.12	25354640	40.5	7.18	1.21	6.6	1.12
Of Strange Lands and People, Scenes From Child	18611324	5245224	28.2	12.35	7.02	1.54	0.88	5282460	28.4	2.09	1.19	1.89	1.07
Sonata No 15 in D major, Op. 28 _Pastoral_, Sche	26815208	8134639	30.3	18.23	7.20	2.18	0.86	8022964	29.9	2.79	1.10	2.53	1.00
Symphony No 4, 4th Movement (excerpt).aif	34482630	12887097	37.4	23.43	7.19	3.27	1.00	13172258	38.2	3.92	1.20	3.7	1.14
Symphony No 8 (excerpt).aif	39868698	16697146	41.9	27.22	7.23	4.37	1.16	16871912	42.3	4.8	1.27	4.24	1.13
TOTAL	1.459E+09	652084938	44.7	1009	7.32	148.81	1.08	679168214	46.5	176.82	1.28	166.78	1.21

Table 6d. Results for FLAC with option “-8” and WavPack fast setting.

file name	size	WavPack -h						
		size	%	encode	decode	enc (ls -r)		
00hi.aif Choral	55918906	20219622	36.2	14.18	2.68	8.22	1.56	16.77
01ce.aif Solo cello	177312682	71092468	40.1	32.37	1.93	26.63	1.59	36.14
02be.aif Orchestra	43582666	16844800	38.7	7.98	1.94	6.68	1.62	9.45
03cc.aif Ballet	24395050	13197188	54.1	4.72	2.05	4.21	1.83	5.61
04sl.aif Software synthesis	85029610	34201148	40.2	15.57	1.94	12.54	1.56	17.59
05bm.aif Club techno	59136442	39218860	66.3	11.54	2.07	9.85	1.76	12.7
06eb.aif Rampant trance techno	43998970	28620610	65.0	8.48	2.04	7.54	1.81	9.81
07bi.aif Rock	88853962	50473824	56.8	17.03	2.03	13.92	1.66	19.04
08ky.aif Pop	35896330	24403400	68.0	7.04	2.08	6.3	1.86	8.51
09sr.aif Indian classical 1	71693770	31453468	43.9	13.39	1.98	11	1.62	17.45
10si.aif Indian classical 2	89411386	44480598	49.7	17.17	2.03	14.01	1.66	18.81
1 kHz t.b.v. Calibration- Left Channel.aif	3880854	974642	25.1	0.95	2.59	0.7	1.91	
1 kHz t.b.v. Calibration- Right Channel.aif	3883064	997094	25.7	0.92	2.51	0.7	1.91	
1_3 Octave Bands of Pink Noise.aif	53451468	16742916	31.3	9.67	1.91	8.04	1.59	
1_3 octave Bands of Pink Noise- Left Channel.aif	53451454	16732726	31.3	9.72	1.92	7.73	1.53	
A Passing Train In the Quiet Dutch Farmlands???.aif	25316894	8138458	32.1	4.66	1.95	3.74	1.56	
Apparatus Musico-Organisticus, Toccata 1a.aif	57685084	29298970	50.8	10.67	1.96	8.91	1.63	
Baiao Malandro.aif	49217864	17381370	35.3	9.48	2.04	7.34	1.58	
Broadbank Pink Noise.aif	3765444	2392636	63.5	0.94	2.64	0.71	2.00	
Capoeira (from Ciclo Nordestino No 3).aif	8563564	2566926	30.0	1.66	2.05	1.37	1.69	
Come Hither You That Love.aif	21523102	9372132	43.5	4.08	2.01	3.39	1.67	
Concerto For 2 Trumpets & Strings, RV 537, C Maj	33694726	15240956	45.2	6.43	2.02	5.09	1.60	
Concerto No 2 in D major, Allegro.aif	38281084	16143934	42.2	7.36	2.03	5.9	1.63	
Everything's Gonna Be All Right.aif	41545622	21611908	52.0	7.96	2.03	6.47	1.65	
Hangin' On To The Good Times.aif	50805440	26244744	51.7	9.76	2.03	8.2	1.71	
Lute Solo.aif	16760254	5393968	32.2	3.2	2.02	2.54	1.60	
Martelo (from Ciclo Nordestino No. 1).aif	7674506	2687014	35.0	1.51	2.08	1.16	1.60	
Nevermore (excerpt).aif	32107062	12383110	38.6	6.19	2.04	4.86	1.60	
Northern Lights.aif	62624250	24114800	38.5	11.45	1.94	9.58	1.62	
Of Strange Lands and People, Scenes From Childh	18611324	5011376	26.9	3.62	2.06	2.76	1.57	
Sonata No 15 in D major, Op. 28 _Pastoral_, Schei	26815208	7646346	28.5	4.83	1.91	3.92	1.55	
Symphony No 4, 4th Movement (excerpt).aif	34482630	12683546	36.8	6.58	2.02	5.38	1.65	
Symphony No 8 (excerpt).aif	39868698	16349608	41.0	7.36	1.95	6.14	1.63	
TOTAL	1.459E+09	644315166	44.2	278.47	2.02	225.53	1.64	

Table 6e. Results for WavPack high compression setting and files in reverse order.